IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 8 December 2025, accepted 20 December 2025, date of publication 30 December 2025, date of current version 8 January 2026.

Digital Object Identifier 10.1109/ACCESS.2025.3649730

RESEARCH ARTICLE

HARMONY: A Holistic Auto-Scaling Resilient Model for Multi-Objective Adaptation in Microservices Using HPPNs–A Case Study on E-Business Platforms

SOUHEIR MERKOUCHE[1], CHAFIA BOUANAKA[1], ELHADJ BENKHELIFA[2], (Senior Member, IEEE), AND CHIRINE GHEDIRA GUEGAN[3]

[1]LIRE Laboratory, University of Constantine 2-Abdelhamid Mehri, Constantine 25016 , Algeria
[2]Smart Systems, AI and Cybersecurity Research Center, University of Staffordshire, ST4 2DE Stoke-on-Trent, U.K.
[3]iaelyon School of Management, INSA Lyon, LIRIS, UMR5205, University of Lyon 3, 69008 Lyon, France

Corresponding author: Souheir Merkouche (souheir.merkouche@univ-constantine2.dz)

ABSTRACT Modern microservice architectures face significant auto-scaling challenges due to dynamic workloads, heterogeneous resource demands, and competing quality objectives like performance, cost, and reliability. Existing solutions often adopt either centralized (bottleneck-prone) or decentralized (conflict-prone) architectures, rely on single-metric threshold-based adaptations, and lack formal guarantees on system behavior. To address these limitations, we propose HARMONY, a holistic auto-scaling framework that ensures both hierarchical coordination, multi-objective adaptation, and formal verification for microservices. The layered MAPE-K architecture of HARMONY enables coordinated adaptation between individual microservices and the overall system through shared knowledge and distributed quality adaptation mechanisms. The Pareto-based planning strategy of HARMONY dynamically selects compromise scaling actions that balance system-wide objectives with microservice-level constraints. HARMONY formalizes adaptation behaviors using Hierarchical Parallel Petri Nets (HPPNs), enabling rigorous verification of deadlock freeness, boundedness, and correctness. We implement HARMONY through the ScopeNet framework and evaluate it through a realistic e-business case study. Results show that HARMONY: 1) autonomously coordinates cross-service adaptations, 2) optimizes trade-offs among conflicting objectives, and 3) ensures reliable behavior under diverse runtime conditions. These capabilities represent a significant improvement in the state of the art for scalable, verifiable, and adaptive microservice auto-scaling.

INDEX TERMS Microservices, auto-scaling, cloud computing, formal verification, Petri nets.

I. INTRODUCTION

Cloud-native applications and service-oriented architectures have become the backbone of modern digital platforms, supporting services at unprecedented scale and complexity [1]. Microservices particularly enabled fine-grained deployment, elastic resource management, and independent scaling of service components. Nonetheless, such flexibility introduces significant challenges in auto-scaling management. Microservices operate under dynamic workloads, heterogeneous resource demands, and diverse quality requirements such as performance, cost efficiency, and reliability [2]. Ensuring that microservices adapt effectively to fluctuating runtime conditions requires complex, coordinated auto-scaling strategies.

The auto-scaling challenge in microservices-based architectures is inherently multi-dimensional, encompassing both architectural design considerations and dynamic adaptation strategies. From an **architectural** perspective, most existing auto-scaling solutions are either fully centralized, leading to scalability bottlenecks and delayed decision-making, or fully decentralized, resulting in conflicting adaptations and

The associate editor coordinating the review of this manuscript and approving it for publication was Nitin Gupta.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

inconsistent global behavior [3]. Few approaches offer an inherently hierarchical system structure that simultaneously supports local autonomy and ensures global coherence across services.

At the **adaptation strategy** level, many current solutions [2], [4] rely on static threshold-based mechanisms that react to local metric violations, such as CPU or memory thresholds. While threshold-based strategies are practical and widely used, their typical implementations focus on single objectives and neglect the multi-criteria aspects of service quality. More recent solutions [5], [6] address multiple metrics but often lack formal verification or clear coordination mechanisms.

Modern auto-scaling goes beyond local, isolated adaptations: it needs to coordinate adaptations across microservices, balance multiple system quality attributes, and systematically manage trade-offs among conflicting objectives, such as enhancing performance while minimizing cost.

In many microservices-based systems, auto-scaling mechanisms are defined through ad-hoc rules or simulation-based evaluations without rigorous formalization [7]. The lack of formal models limits the capability to anticipate system behavior in complex adaptation scenarios, impedes critical properties verification including deadlock freeness, boundedness, and adaptation correctness, and ultimately undermines the reliability and reproducibility of auto-scaling decisions.

In the present work, we introduce **HARMONY** (Holistic Auto-scaling Resilient Model for Multi-Objective Adaptation in Microservices), a unified auto-scaling framework that integrates architectural hierarchy, multi-objective adaptation strategies, and formal verification into a single coherent model. From an architectural perspective, HARMONY adopts a layered MAPE-K design in which global Application Managers (AMs), local Microservice Managers (MSMs), and Quality Analyze-Planners (QAPs) are coordinated through hierarchical decision loops and shared knowledge repositories. Such architecture supports autonomous local reactions while maintaining system-wide coherence.

From an adaptation perspective, HARMONY introduces a Pareto-based, multi-objective scaling strategy that selects minimal yet balanced adaptation plans from candidate microservice proposals. HARMONY applies a multi-criteria adaptation process that reconciles the quality requirements of individual microservices with system-wide objectives, even though the considered qualities are often conflicting such as performance, availability and cost.

To ensure the correctness of the auto-scaling decisions, we formalize HARMONY's adaptation approach using Hierarchical Parallel Petri Nets (HPPNs), which inherently support hierarchical interactions, parallel execution, and complex token structures. These capabilities make HPPNs particularly well-suited for modeling the proposed hierarchical auto-scaling architecture and implementing the multi-objective adaptation strategies introduced in this work. The resulting formal model enables systematic simulation and verification of complex adaptive behaviors, offering guarantees on critical system properties such as deadlock-freeness and boundedness, and ensuring reliability of adaptation decisions.

We validate HARMONY through a concrete case study on an e-business platform composed of heterogeneous microservices. The results show its ability to model service-level adaptations, verify critical structural and behavioral properties, and dynamically select optimal scaling actions under varying and potentially unpredictable runtime conditions.

The remainder of this paper is organized as follows. Section II reviews related work on microservice auto-scaling architectures and strategies. Section III introduces the HARMONY architecture, describing its hierarchical coordination structure and decision-making layers. Section IV details the proposed approach, focusing on its multi-objective auto-scaling logic and operational mechanisms. Section V formalizes the adaptation logic using HPPNs and describes its implementation within the ScopeNet framework. Section VI presents a case study on a realistic e-business platform, demonstrating the model's behavior and effectiveness under diverse adaptation scenarios. Finally, Section VII concludes the paper and highlights directions for future research.

II. RELATED WORK

Auto-scaling has emerged as a fundamental enabler of elasticity, efficiency, and resilience in microservices-based cloud environments. However, existing solutions differ significantly in their architectural designs, adaptation strategies, and verification methods. This section reviews key categories of prior work, with an emphasis on auto-scaling architectures, adaptation strategies, multi-objective scaling techniques, and formal modeling efforts in microservices-based systems. The objective is to identify their weaknesses that HARMONY addresses through a unified, hierarchical, multi-objective, and formally verified auto-scaling framework.

A. AUTO-SCALING ARCHITECTURES FOR MICROSERVICES

The architectural design of auto-scaling systems significantly influences their scalability, responsiveness, and coordination effectiveness. Thus, three main classes of auto-scaling architectures can be distinguished: centralized, decentralized, and hybrid.

Centralized controllers, exemplified by Kubernetes' Horizontal Pod Autoscaler [8], AWS Application Auto Scaling [9], and Smart HPA [3], provide elementary coordination mechanisms through a single control plane. In such systems, a central controller continuously monitors the overall system state and makes all scaling decisions.

Although centralized architectures simplify coordination and provide a global view of the system state, they introduce potential bottlenecks, scalability limitations, and single points of failure. As demonstrated by Google's Borg cluster analysis [10], centralized controllers such as Kubernetes HPA exhibit exponential latency growth beyond 500 nodes, with decision times exceeding 1.8 s at scale. In contrast, decentralized auto-scaling architectures like

Hydra [11] and Erlang/OTP's distributed autoscalers [12] distribute decision-making across microservices, allowing each component to scale independently [4], [6]. This approach improves scalability and local responsiveness but can lead to inconsistent adaptations and conflicts between local and global objectives due to the lack of system-wide coordination. Recent edge-computing studies report 38% higher SLA violations in decentralized systems compared to their centralized counterparts [13].

To address these limitations, hybrid architectures such as FIRM [14] and hierarchical Kubernetes extensions [15] have been proposed. These architectures typically feature multi-level controllers, where local microservice managers handle immediate adaptation and higher-level controllers coordinate global objectives. Despite their potential, hierarchical architectures remain underexplored and often lack formal mechanisms for cross-layer and multi-objective coordination.

B. AUTO-SCALING STRATEGIES FOR MICROSERVICES

Several auto-scaling strategies have been proposed to manage resources and maintain service quality in microservices-based architectures. In this subsection, we present representative works, explain their core contributions, and highlight their limitations with respect to coordinated, multi-objective, and formally verifiable adaptation.

Rossi et al. [16] proposed a hierarchical auto-scaling framework for Kubernetes that distributes decision-making across multiple levels of the system. Although the framework enhances scalability and responsiveness, it neither explicitly addresses trade-offs between competing quality attributes such as performance and cost nor provides mechanisms for multi-criteria coordination.

Alexander et al. [17] proposed a cost-aware orchestration approach for heterogeneous cloud platforms, prioritizing resource allocation strategies designed to minimize operational costs. Although effective in reducing costs, the approach exhibits a bias toward cost optimization that can inadvertently degrade other critical system qualities, such as performance and reliability.

Xiao and Hu [18] developed DScaler, a deep reinforcement learning-based autoscaler that dynamically adjusts resource allocation in response to workload fluctuations. While DScaler effectively improves performance and adapts to changing load conditions, its design primarily emphasizes horizontal scaling and does not explicitly account for broader system qualities or multi-objective decisions.

Meng et al. [5] introduced DeepScaler, which leverages spatiotemporal graph neural networks to predict resource demands and proactively scale microservices. Although DeepScaler enhances workload prediction and scaling responsiveness, it predominantly targets performance optimization and pays limited attention to multi-objective considerations or cross-service coordination.

Yuan and Liao [19] employed time-series forecasting to enable elastic scaling in Kubernetes clusters. Their solution efficiently predicts resource needs but lacks mechanisms for quality-specific coordination among services and does not account for conflicting adaptation goals.

Ahmad et al. [3] proposed Smart HPA, a hybrid autoscaler that dynamically alternates between local and centralized scaling decisions according to system resource availability. While Smart HPA mitigates decision bottlenecks and adapts effectively to resource constraints, its primary focus remains on utilization thresholds, and it lacks a structured framework for multi-objective adaptation.

Santos et al. [2] proposed MS-RA, a requirements-driven autoscaler that supports reactive, threshold-based scaling aligned with service-level objectives (SLOs). While MS-RA improves local adaptation based on quality requirements, it does not explicitly coordinate scaling actions across services or address system-wide trade-offs.

Xie et al. [4] presented PBScaler, a bottleneck-aware autoscaler that identifies and scales critical microservices based on observed performance degradation. While PBScaler effectively detects and mitigates local bottlenecks, it lacks a comprehensive global adaptation strategy and does not account for the systemic impact of local scaling decisions.

Horn et al. [6] proposed MOHA, a multi-objective hybrid autoscaler that applies Pareto-based plan selection in Kubernetes environments. MOHA integrates local and global objectives but does not explicitly support hierarchical control or formal coordination across multiple adaptation layers.

Dashtbani and Tahvildari [20] designed STaleX, a spatiotemporal PID-controller framework per microservice complemented by a supervisory unit for coordinated scaling. Although it improves SLO adherence and coordination, it does not provide formal mechanisms for multi-plan optimization or explicit management of quality trade-offs.

Naik et al. [21] defined a Petri net-based model for fault detection in microservices, validating error propagation and recovery behaviors. While the approach is effective for reliability analysis, it does not provide formal support for auto-scaling logic or adaptive plan selection under QoS constraints.

We conclude that most existing strategies:

- focus on either local thresholds or global objectives, without integrating both hierarchically;
- prioritize a single adaptation goal, such as cost or performance, without balancing conflicting system qualities; and
- lack formal verification to guarantee the correctness and reproducibility of scaling decisions.

To address these limitations, we propose a hybrid architecture that combines local autonomy with global coordination through structured knowledge sharing. Unlike prior approaches, HARMONY explicitly synchronizes adaptation processes across layers and systematically resolves conflicts between local service objectives and system-wide scaling goals. It integrates threshold-based detection with a hierarchical, multi-criteria adaptation strategy that selects

TABLE 1. Comparison of auto-scaling architectures and strategies for microservices.

Approach	Architecture	Multi-Objective Support	Cross-Service Coordination	Formal Verification	Limitations
Kubernetes HPA [8]	Centralized	✗	✗	✗	Scalability bottlenecks at large scale
AWS App Auto Scaling [9]	Centralized	✗	✗	✗	Single control plane, limited flexibility
Smart HPA [3]	Centralized	✗	✗	✗	Threshold-based, limited multi-objective support
Hydra [11]	Decentralized	✗	✗	✗	Lacks system-wide coordination
Erlang/OTP [12]	Decentralized	✗	✗	✗	High SLA violation rates, local-only views
PBScaler [4]	Decentralized	✗	✗	✗	No global strategy, local bottleneck detection only
MOHA [6]	Decentralized	✓	✗	✗	No hierarchical coordination, local-only adaptation
FIRM [14]	Hybrid	Partial	Partial	✗	Limited cross-layer coordination
Hierarchical Kubernetes [15]	Hybrid	Partial	Partial	✗	No formal conflict resolution
Rossi et al. [16]	Hybrid	✗	Partial	✗	No multi-objective or formal plan coordination
Alexander et al. [17]	Centralized	Partial (Cost-Focused)	✗	✗	Cost-focused, other qualities sacrificed
Xiao and Hu (DScaler) [18]	Centralized	✗	✗	✗	Focuses on horizontal scaling, ignores trade-offs
Meng et al. (DeepScaler) [5]	Centralized	✗	✗	✗	Performance-centric, no multi-objective coordination
Yuan and Liao [19]	Centralized	✗	✗	✗	Predicts resource needs, no quality coordination
Santos et al. (MS-RA) [2]	Centralized	Partial (SLO-Aware)	✗	✗	Lacks cross-service planning
STaleX [20]	Hybrid	Partial (SLO-Aware)	Partial	✗	No formal plan selection, limited multi-plan coordination
Naik et al. [21]	Centralized	✗	✗	Partial (Fault Modeling)	Does not support formal auto-scaling logic
HARMONY	Hybrid	✓	✓	✓	Fully integrated multi-layer, multi-objective, formally verified system

coordinated adaptation plans, balances local and global objectives, and supports formal verification via HPPNs.

III. HARMONY HYBRID ARCHITECTURE

The proposed architecture, see Figure 1, introduces a formally founded and operationally viable model for auto-scaling in microservices-based systems. It deploys MAPE-K (Monitor–Analyze–Plan–Execute–Knowledge) feedback loops across three interconnected layers, each responsible for adaptation at a distinct scope of control. The hierarchical design of HARMONY supports scalable and modular adaptation, allowing layers to operate independently while coordinating when necessary. This hierarchy enforces separation of concerns while preserving the flexibility to orchestrate actions across abstraction levels. Dotted with a shared knowledge repository, HARMONY achieves resilient, multi-objective adaptation in dynamic cloud environments, where rapid and well-informed decision-making is essential.

A. APPLICATION MANAGER

The Application Manager (AM) oversees the system's global behavior, ensuring adherence to high-level quality objectives such as cost efficiency, responsiveness, and throughput. Functioning as a centralized MAPE-K control loop, it monitors system-wide metrics, coordinates adaptation actions across microservices, and triggers global scaling when thresholds are violated. The AM synthesizes responses from lower layers into coherent execution plans, reconciling local actions with system-wide goals.

B. MICROSERVICE MANAGERS

Each Microservice Manager (MSM) functions autonomously, controlling a unique microservice through an independent MAPE-K loop that monitors local quality attributes such as latency and performance. MSMs continuously analyze runtime metrics to identify adaptation requirements while remaining responsive to directives from the Application Manager. By reconciling local priorities with system-wide objectives, MSMs enable fine-grained yet coordinated scaling decisions.

C. QUALITY ANALYZE-PLAN

Embedded within each MSM, the Quality Analyze-Plan (QAP) modules focus on specific quality requirements. They generate adaptation proposals in response to local metric deviations or breaches of global thresholds. The MSM then consolidates the outputs from QAPs into executable adaptation plans, which can be applied locally or forwarded to higher-level controllers for coordinated system-wide action.

D. SHARED KNOWLEDGE

The HARMONY architecture differs from conventional MAPE-K implementations by incorporating shared knowledge across its hierarchical layers. Such shared repository maintains real-time system states, deployment configurations, quality thresholds, historical observations, and pending adaptation plans. As a result, the architecture facilitates coherent decision-making, resolves conflicts between local

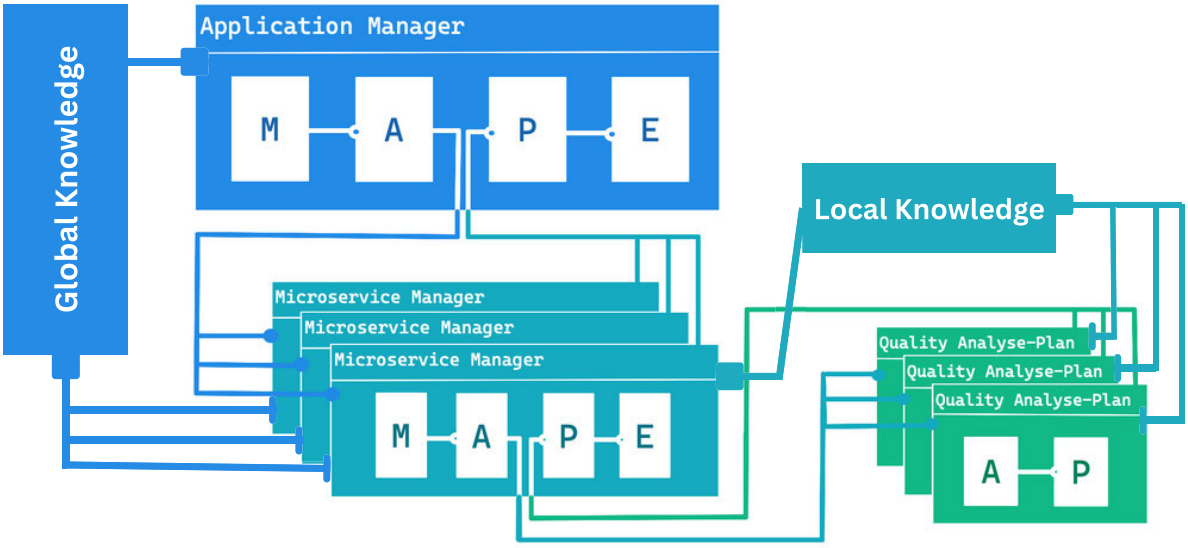


FIGURE 1. The hybrid auto-scaling architecture in HARMONY.

and global objectives, and enables reproducible and explainable adaptation processes.

IV. A MULTI-OBJECTIVE AUTO-SCALING APPROACH FOR HARMONY

The auto-scaling approach of HARMONY tackles a key challenge in microservice architectures: balancing competing quality objectives (e.g., cost, latency, throughput) across decentralized services while maintaining system-wide stability. HARMONY adopts a hierarchical decision-making approach, aligned with its three-layer architecture, to dynamically reconcile local and global requirements. The approach leverages continuous monitoring, shared system knowledge, and Pareto-optimal compromise selection to ensure that adaptations are both responsive to runtime fluctuations and aligned with high-level policies. By decentralizing decision-making without sacrificing coordination, HARMONY achieves scalable multi-objective optimization in dynamic cloud environments.

A. GLOBAL DECISION-MAKING OF HARMONY

The Application Manager (AM) functions as the central authority responsible for maintaining system-wide quality objectives in HARMONY. Its primary role is to detect violations of global metrics, such as aggregated CPU usage, total memory consumption, or overall system input rate, and to orchestrate coordinated scaling actions that restore system stability. When a metric exceeds its predefined threshold (e.g., CPU utilization surpassing 85%), the AM initiates a global adaptation cycle, launching a two-phase process to resolve the violation while simultaneously balancing competing quality objectives.

1) PHASE 1: ADAPTATION PLANS AGGREGATION AND STRUCTURING

The AM issues an auto-scaling request to all MSMs, prompting each to submit a candidate adaptation plan. Each plan is represented as a tuple comprising:

- **Microservice identifier** (ms_i): Uniquely labels the originating service
- **Replica adjustment proposal** (k_i): Specifies the number of replicas to add/remove (e.g., $k_i = +2$)
- **Metric impact estimate** (s_i): Quantifies the expected improvement (e.g., "15% CPU reduction")
- **Priority flag** (p_i): A Boolean indicating whether the plan addresses a *local QoS violation* (p_i = true) or contributes to *global optimization* (p_i = false)

To enable multi-objective adaptation, HARMONY employs a Pareto-based optimization strategy that identifies a compromise subset of adaptation plans. Rather than evaluating individual plans, the model examines all possible subsets of the candidate plans to determine which combinations collectively satisfy the global adaptation objective (e.g., a 20% reduction in CPU usage).

Each plan is associated with a quantified impact on multiple quality attributes (such as cost, performance, or availability) represented as a multi-dimensional score. For every subset whose cumulative adaptation impact satisfies or exceeds the global requirement, the model calculates aggregate quality scores. Among the valid subsets, it filters out those that are dominated in the Pareto sense.

If multiple non-dominated (Pareto-optimal) subsets remain, a compromise selection is performed using a weighted aggregation of their quality impacts. This ensures that the final subset of plans not only restores system compliance but also balances trade-offs across competing objectives.

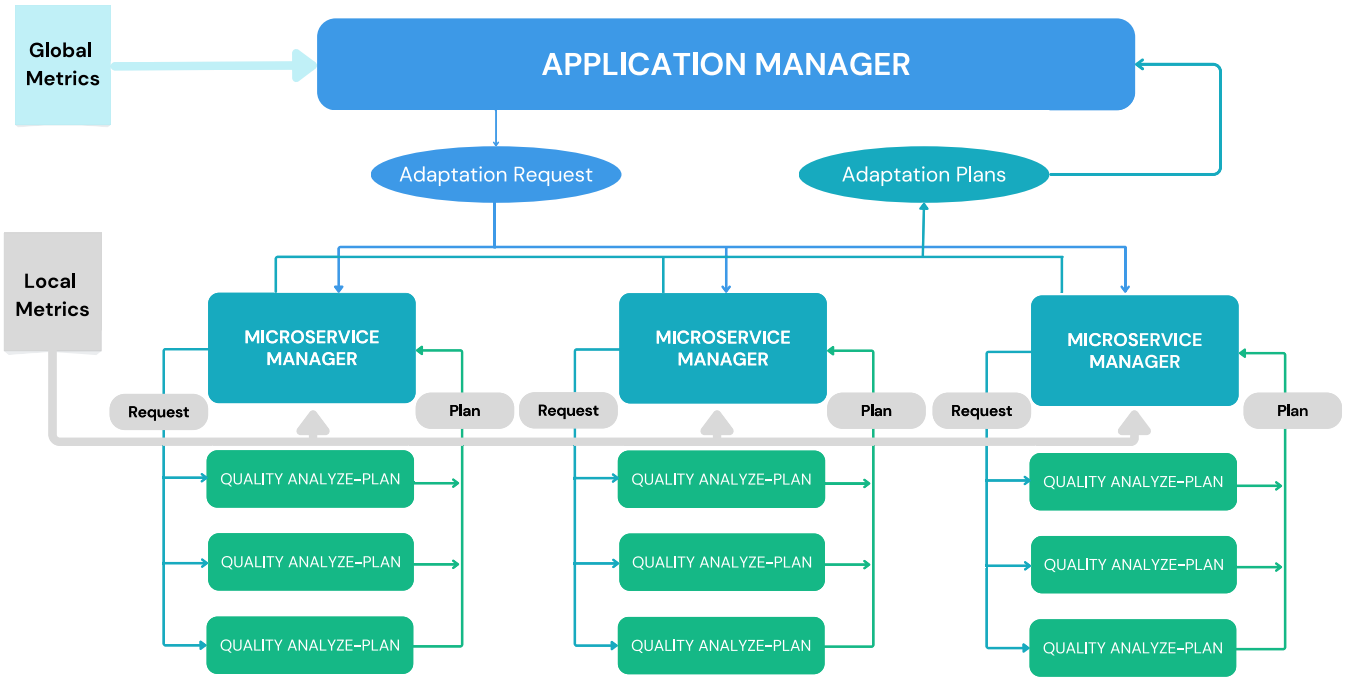


FIGURE 2. The HARMONY multi-objective auto-scaling approach.

2) PHASE 2: PARETO-OPTIMAL PLAN SELECTION

In this phase, the AM synthesizes a compromise adaptation plan using Pareto-based optimization [22]. Given a set of candidate plans $\mathcal{P} = \{P_1, \ldots, P_n\}$ from MSMs, the AM:

1) **Computes Multi-Dimensional Effects**: For each plan P_i, evaluates its impact on all system qualities (e.g., cost, latency, throughput) through the function:

$$\text{Effect}(P_i) = \left(\Delta Q_1^{P_i}, \Delta Q_2^{P_i}, \ldots, \Delta Q_m^{P_i}\right) \quad (1)$$

where $\Delta Q_j^{P_i}$ quantifies the change in quality Q_j if P_i is executed.

2) **Identifies Valid Combinations**: Generates plan combinations $\mathcal{C} \subseteq 2^{\mathcal{P}}$ that restore the violated metric Q_v to its target range:

$$\exists\, \mathcal{C} \text{ s.t. } M^{\text{current}} + \sum_{P_i \in \mathcal{C}} \Delta M^{P_i} \in \left[M^{\min}, M^{\max}\right] \quad (2)$$

3) **Constructs Pareto Front**: For each valid $\mathcal{C}$, computes the aggregate quality vector:

$$\Delta \mathbf{Q}^{\mathcal{C}} = \left(\sum_{P_i \in \mathcal{C}} \Delta Q_1^{P_i}, \ldots, \sum_{P_i \in \mathcal{C}} \Delta Q_m^{P_i}\right) \quad (3)$$

then filters non-dominated solutions where no other $\mathcal{C}'$ improves any Q_j without worsening another Q_k ($j \neq k$).

4) **Selects a compromise solution**: Chooses the smallest $\mathcal{C}^*$ from the Pareto front that:
   - Satisfies (2),
   - Prioritizes plans addressing local violations ($p_i = \text{true}$),
   - Minimizes $|\mathcal{C}^*|$ (fewest executed plans).

This approach ensures:

- **Corrective Guarantee**: The violated metric is always restored.
- **Resource Efficiency**: No redundant scaling actions are selected.
- **Quality Preservation**: qualities degrade only if unavoidable.

This process ensures that adaptation at the global level is not only effective in correcting violations, but is also balanced across quality dimensions and services, preserving system-wide stability and fairness.

The overall decision process is implemented through **Algorithm** 1 (GlobalPlanSelector), which identifies valid subsets and invokes **Algorithm** 2 (SelectParetoOptimalPlans) to filter and select the compromise plan.

Example: Suppose the system detects a CPU violation of 20% and receives one adaptation plan from each of five microservices. Each plan offers a partial gain (e.g., 5%, 8%, 3%, etc.) and impacts other qualities like cost and performance. The AM needs to select a subset whose combined gain reaches or exceeds 20% but also offers the best trade-off between increasing performance and minimizing cost. For instance, selecting plans from services A, B, and C may achieve 21% gain, reduces cost by 4%, and improves performance by 5%, while another subset achieves the same CPU gain but at higher cost. The chosen set $\mathcal{C}^*$ is the minimal, Pareto-optimal compromise set that balances these objectives.

Algorithm 1 GlobalPlanSelector - Select Adaptation Plans

Require: Set of plans $P = \{p_1, \ldots, p_n\}$, adaptation goal θ
Ensure: Subset of plans $P_{\text{sel}} \subseteq P$ satisfying the goal

$S_{\text{valid}} \leftarrow \emptyset$
for all $S \subseteq P$ **do**
  if $\sum_{p \in S} \text{Gain}(p) \geq \theta$ **then**
    $S_{\text{valid}} \leftarrow S_{\text{valid}} \cup \{S\}$
  end if
end for
if $S_{\text{valid}} = \emptyset$ **then**
  return $\emptyset$
end if
return SelectParetoOptimalPlans(S_{valid})

Algorithm 2 SelectParetoOptimalPlans - Filter by Multi-Objective Dominance

Require: S_{valid}: Set of valid plan subsets
Ensure: A compromise subset P_{sel}

ParetoSet $\leftarrow \emptyset$
for all $S_i \in S_{\text{valid}}$ **do**
  dominated $\leftarrow$ false
  for all $S_j \in S_{\text{valid}}$ such that $S_j \neq S_i$ **do**
    if S_j dominates S_i **then**
      dominated $\leftarrow$ true
      break
    end if
  end for
  if not *dominated* **then**
    ParetoSet $\leftarrow$ *ParetoSet* $\cup \{S_i\}$
  end if
end for
if $|\textit{ParetoSet}| = 1$ **then**
  return the single solution in *ParetoSet*
else
  Use a scoring function (e.g., weighted sum) to select the best compromise
  return best-scoring subset from *ParetoSet*
end if

B. LOCAL DECISION-MAKING

As formalized in Algorithm 3, each MSM autonomously monitors its microservice through resource utilization metrics (e.g., CPU and Memory) and initiates its adaptation process whenever:

- A local metric violates its threshold (e.g., CPU > 80%), or
- A global scaling request is received from the AM

When the adaptation process is launched, each QAP module Q_i generates a plan tuple:

$$P_i = (k_i, q_i) \tag{4}$$

where:

- k_i: Proposed replica change

Algorithm 3 MSM Adaptation Decision Process

Require: Set of QAP modules $\{Q_1, \ldots, Q_m\}$

$\mathcal{P} \leftarrow \emptyset$
for each quality attribute $j \in \{1, \ldots, m\}$ **do**
  $P_j \leftarrow Q_j$.generatePlan() ▷ Plan with k_j replicas
  $\mathcal{P} \leftarrow \mathcal{P} \cup \{P_j\}$
end for
$k^* \leftarrow \left\lfloor \frac{\sum_{i=1}^{R} k_i \cdot q_i}{R} \right\rfloor$ ▷ Weighted replica count
if trigger == local violation **then**
  executeScaling(k^*)
else
  submitToAM(k^*)
end if

Algorithm 4 Scale-to-Satisfy

$k \leftarrow$ currentReplicas()
repeat
  $k' \leftarrow k + 1$
  $m' \leftarrow$ predictMetric(k') ▷ Projected metric value
  $q' \leftarrow$ predictQuality(k') ▷ Projected quality value
  if inRange(m') $\vee$ $\neg$isValid(q') **then**
    break
  end if
  $k \leftarrow k'$
until maxIterations
return $(k - 1, q')$

- q_i: Importance rate of the quality attribute (normalized [0, 1])

The MSM then computes the final adaptation plan using:

$$SF = \frac{\sum_{i=1}^{R} k_i \times q_i}{R} \tag{5}$$

where R is the total number of quality attributes (QAPs).

Through this compromise-based process, each MSM attains autonomy for immediate local responses, cooperation via conditional deferral to global coordination, and adaptivity through dynamic weight adjustments informed by time-based policies, SLA overrides, or reinforcement learning.

C. QUALITY-CENTRIC PLANNING

Each QAP module executes adaptation through two distinct strategies which are:

1) SCALE-TO-SATISFY

Upon receiving a local adaptation request, the QAP executes a Scale-to-Satisfy strategy, in which it generates the adaptation plan by incrementally increasing or decreasing the number of replicas. For each step, it:

- Predicts new metric value m' (e.g., CPU usage)
- Predicts new quality value q' (e.g., cost)

The process is iterated until either:

- Metric m' is scaled back to the targeted range **or**
- Quality q' would violate its constraints

Algorithm 5 Scale-to-Optimize

$k \leftarrow$ currentReplicas()
repeat
  $k' \leftarrow k + 1$
  $m' \leftarrow$ predictMetric(k')
  $q' \leftarrow$ predictQuality(k')
  if $\neg$inRange(m') $\vee$ $\neg$isValid(q') **then**
    break ▷ Terminate when constraints fail
  end if
  $k \leftarrow k'$
until maxIterations
return $(k-1, q')$

The strategy is implemented in Algorithm 4.

2) SCALE-TO-OPTIMIZE

When receiving a global adaptation request, the QAP executes a Scale-to-Optimize strategy, in which the increment/decrement process continues as long as:

- Metric m' remains within its target range **and**
- Quality q' satisfies its constraints

The QAP outputs the final valid plan that maximizes quality improvements while satisfying all system constraints. The process is implemented in Algorithm 5.

Figure 2 captures the workflow of our Multi-Objective Auto-Scaling Approach, showcasing the coordinated decision-making between the AM, MSMs, and QAP modules. The hierarchical interaction enables balanced adaptations that reconcile system-wide objectives with microservice-level quality constraints. In the next section, we formally model this approach using HPPNs.

V. HPPN-BASED FORMAL MODEL FOR HARMONY

To support a rigorous representation, simulation, and verification of the auto-scaling approach in HARMONY, we adopt the HPPN formalism, implemented via the ScopeNet framework. The formal model mirrors HARMONY's layered architecture and captures its multi-objective decision-making processes across autonomous yet coordinated control loops.

A. BACKGROUND ON THE HIERARCHICAL PARALLEL PETRI NETS AND ScopeNet

In this section, we recall the foundational elements of the HPPN formalism and the ScopeNet framework used for modeling and verifing the auto-scaling approach of HARMONY.

1) THE HIERARCHICAL PARALLEL PETRI NETS

HPPNs are an extension of Petri nets to model complex, modular, and adaptive systems with layered control structures. They enable representating hierarchical architectures composed of autonomous yet cooperative components.

An HPPN consists of a *hierarchy of nets*, organized as a tree structure. The *parent net* coordinates a set of *parallel child nets*, each of which may recursively serve as a parent to lower-level nets. Such structure reflects the layered control patterns where global, local, and quality-specific decisions operate in coordination.

The key mechanism enabling communication between hierarchical layers is the *portal transition*, which governs the bidirectional flow of tokens. When a portal fires, it triggers two phases:

- **Firing-in:** Tokens are distributed from the parent net to each child net via specific input transitions.
- **Firing-out:** After local processing, child nets return updated tokens, which are consolidated into a compound token and reintroduced into the parent net.

The structure of HPPNs enables *parallel execution*, *cross-level coordination*, and *structured token exchange* in a way that mirrors hierarchical decision-making processes.

Definition 1: An HPPN is formally defined as a triplet

$$HPPN = (PN, PT, F)$$

where:

- PN is a set of parallel nets;
- PT is a set of portal transitions;
- $F \subseteq (PN \times PT) \cup (PT \times PN)$ is a set of arcs connecting parallel nets to portal transitions.

Definition 2: Each *parallel net* $PN \in PN$ is defined as:

$$PN = (P, T, F, A, W, D, Type, M_0)$$

with:

- $P = P^{(c)} \cup P^{(f)}$ a set of places, including *complex places* $P^{(c)}$ and *fusion places* $P^{(f)}$;
- $T = T^{(h)} \cup T^{(f)} \cup T^{(c)}$ a set of transitions, including *high-level transitions*, *fission transitions*, and *computing transitions*;
- $F \subseteq (P \times T) \cup (T \times P)$ a set of directed arcs;
- A is a set of expressions defining arc weights;
- $W : F \to A$ a weight function (default is 1);
- D is a non-empty finite set of token domains (types);
- $Type : P \cup T \to D$ a typing function;
- $M_0 \in \mu(Place)$ the initial marking, with $\mu(Place)$ being multisets over the set $\{(p, g) \mid p \in P, g \in Type(p)\}$.

Definition 3: A *portal transition* $t_p \in PT$ connects a parent net $pn^{\uparrow}$ to a set of child nets $\{pn_i^{\downarrow}\}_{i=1}^{k}$. It is defined by:

$$t_p = (I, O, C_{\text{in}}, C_{\text{out}}, W, F_{\text{in}}, F_{\text{out}})$$

where:

- $I \subseteq P^{\uparrow}$ and $O \subseteq P^{\uparrow}$ are the parent-side input and output places;
- $C_{\text{in}}(p, \tau)$ and $C_{\text{out}}(o, \sigma)$ are boolean arc conditions governing when tokens may be consumed and produced;
- $W : (I \times \{t_p\}) \cup (\{t_p\} \times O) \to \mathbb{N}$ is the arc multiplicity function (default $W = 1$);
- F_{in} maps the selected parent token to the designated portal-input places of each child net;

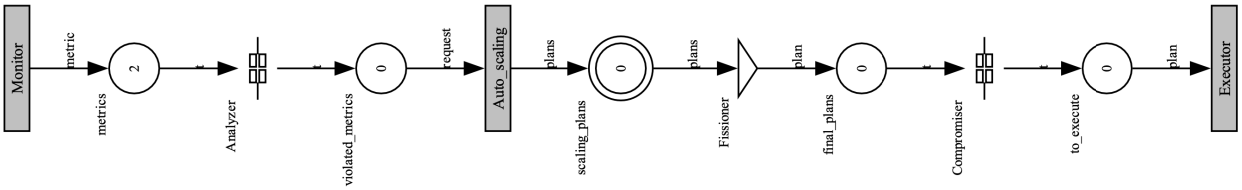


FIGURE 3. Application manager formal model.

- F_{out} aggregates tokens returned from the child nets into a compound token to be returned to the parent net.

Tokens in HPPNs are either *complex tokens*, which are structured typed units, or *compound tokens*, which are collections of complex tokens. This supports fine-grained representation of deployment states, adaptation plans, and quality metrics. The Operational semantics of HPPNs is defined via the firing function associated with each type of transitions in HPPNs.

a: PRELIMINARIES

The marking function in HPPNs is $M : P \to \mu(D)$, $M(p)$ is associates a multiset of tokens of type Type(p)). For a transition $t \in T$, $\bullet t = \{p \in P \mid (p,t) \in F\}$ and $t\bullet = \{p' \in P \mid (t,p') \in F\}$ are its input and output places. Arc weights are given by $W : F \to A$, where numeric weights are interpreted as multiplicities. Multiset subtraction/addition are written $\ominus$, $\oplus$; multiset inclusion as $\preceq$.

b: HIGH-LEVEL TRANSITION ($T \in T^{(H)}$)

Each $t \in T^{(h)}$ carries a guard predicate G_t over the data of input tokens. A *selection* for t at M is a family of multisets $S = \{S_p\}_{p\in\bullet t}$ with $S_p \preceq M(p)$ and $|S_p| = W(p,t)$ (typed by Type(p)). The transition t is *enabled* at M iff there exists a selection S such that

$$G_t\big(\{S_p\}_{p\in\bullet t}\big) = \texttt{true}.$$

When t fires under selection S, the new marking M' is

$$M'(p) = \begin{cases} M(p) \ominus S_p & \text{if } p \in \bullet t \setminus t\bullet, \\ M(p) \ominus S_p \oplus \mathrm{Out}_t(p,S) & \text{if } p \in \bullet t \cap t\bullet, \\ M(p) \oplus \mathrm{Out}_t(p,S) & \text{if } p \in t\bullet \setminus \bullet t, \\ M(p) & \text{otherwise,} \end{cases}$$

where $\mathrm{Out}_t(p,S)$ denotes the multiset of tokens produced on arc (t,p). If an arc expression is defined in A, it is applied to the selected input tokens; otherwise, tokens are forwarded unchanged and replicated according to $W(t,p)$.

c: COMPUTING TRANSITION ($T \in T^{(C)}$)

Each $t \in T^{(c)}$ encapsulates a (deterministic or stochastic) computation f_t that transforms input data. A selection $S = \{S_p\}_{p\in\bullet t}$ is well-typed if $S_p \preceq M(p)$ and $|S_p| = W(p,t)$ for all $p \in \bullet t$. The transition t is *enabled* at a marking M iff such a well-typed selection S exists (and any precondition inside f_t holds). Upon firing, input tokens are consumed and output tokens are computed component-wise by f_t:

$$\big(\mathrm{Out}_t(p',S)\big)_{p'\in t\bullet} = f_t\big(\{S_p\}_{p\in\bullet t}\big),$$

and the marking is updated as above, replacing Out_t accordingly. f_t may aggregate, filter, or rewrite token payloads (e.g., analysis, plan computation), where Type(p') maintained.

d: FISSION TRANSITION ($T \in T^{(F)}$)

Let Comp denotes the type of *compound tokens* (finite multisets or tuples of complex tokens), and $p_{\text{in}} \in \bullet t$ be the input place typed as $\text{Type}(p_{\text{in}}) = \text{Comp}$. Transition t is *enabled* at M iff there exists $\sigma \in M(p_{\text{in}})$ with $\sigma = \langle \tau_1, \ldots, \tau_n \rangle$ $(n \geq 1)$ and $W(p_{\text{in}}, t) \geq 1$.

Firing t consumes the compound token from the input place:

$$M'(p_{\text{in}}) = M(p_{\text{in}}) \ominus \{\sigma\}.$$

Then each constituent complex token is distributed to output places according to π_t:

$$M'(p') = M(p') \oplus \pi_t(p'; \tau_1, \ldots, \tau_n), \qquad \forall p' \in t\bullet.$$

where $\pi_t(p'; \tau_1, \ldots, \tau_n)$ is the multiset of complex tokens delivered to p' (e.g., all τ_i to a single place, or a partition across several places). Types of the produced tokens match Type(p').

e: TYPING AND WEIGHTS

All firings above must satisfy place/transition typing: consumed/produced tokens conform to Type($\cdot$), and multiplicities follow $W(\cdot,\cdot)$. Initial markings M_0 are standard multisets over places, typed as in Def. 2.

f: PORTAL TRANSITIONS ($T_P \in PT$)

Let the parent net be $pn^{\uparrow}$ with marking $M^{\uparrow}$, and the child nets $\{pn_i^{\downarrow}\}_{i=1}^{k}$ with markings $M_i^{\downarrow}$. A portal transition t_p has parent-side input places $I \subseteq P^{\uparrow}$ and output places $O \subseteq P^{\uparrow}$. Each input arc (p,t_p) carries a (boolean) *input arc condition* $C_{\text{in}}(p,\cdot)$, and each output arc (t_p,o) carries an *output arc condition* $C_{\text{out}}(o,\cdot)$; arc weights $W(\cdot,\cdot)$ give multiplicities (default 1).

Enabling (parent side). t_p is enabled at $M^{\uparrow}$ iff there exists an input place $p^* \in I$ and a token $\tau \in M^{\uparrow}(p^*)$ such that

$$C_{\text{in}}(p^*, \tau) = \texttt{true}.$$

TABLE 2. HPPN components and their graphical representation.

Component	Role	Symbol
Complex place: $P^{(c)}$	a place whose tokens carry structured data values. Each token represents a state element, object, or record. The number displayed in the node represents the multiplicity of tokens.	○
Fusion place: $P^{(f)}$	a place whose tokens are compound tokens, i.e., aggregations (tuples or multisets) of complex tokens. Used to group or synchronize multiple state elements. The displayed number represents the count of compound tokens held.	◎
High-level transition: $T^{(h)}$	a transition whose firing is conditioned by a guard predicate over the data values of input tokens.	▬
Computing transition: $T^{(c)}$	a transition that applies a computation or data transformation function to input tokens, producing new output tokens.	▭
Fission transition: $T^{(f)}$	a transition that takes a compound token and decomposes it into its constituent complex tokens, which are then distributed across output places.	▽
Portal transition: $t_p \in PT$	a transition that transfers tokens between hierarchical levels, synchronizing parent and child nets in the HPPN structure.	▭
Marking / Initial marking: M, M_0	a mapping assigning to each place a multiset of tokens. In graphical representations, only token *multiplicity* is shown as a number inside the place.	

We write (p^*, τ) for the selected input.

Phase 1 (Parent → Children). The selected token is consumed from its input place:

$$M^{\uparrow,1}(p^*) = M^{\uparrow}(p^*) \ominus \{\tau\}.$$

The same token is introduced into the portal input of each child:

$$M_i^{\downarrow,1}(q_{\text{in}}) = M_i^{\downarrow}(q_{\text{in}}) \oplus \{\tau\}, \qquad i = 1, \ldots, k.$$

Child execution (parallel). Each child net executes independently according to its own rules until a single token τ_i reaches its portal output interface.

Phase 2 (Children → Parent). The returned tokens are aggregated into one compound token:

$$\sigma = \langle \tau_1, \ldots, \tau_k \rangle.$$

This compound token is produced on each parent output place whose output-arc condition holds, with multiplicity given by the arc weight:

$$\text{for all } o \in O \text{ with } C_{\text{out}}(o, \sigma) = \texttt{true}:$$

$$M^{\uparrow,2}(o) = M^{\uparrow,1}(o) \oplus W(t_p, o) \cdot \{\sigma\}.$$

For clarity, the main HPPN components used in the modeling above and their graphical representations are summarized in Table 2.

2) ScopeNet: MODELING AND VERIFICATION WITH HPPNs

To implement and analyze HPPN-based models, we rely on *ScopeNet*, a Python-based framework that extends the SNAKES Petri net library. ScopeNet supports editing, execution, and verification of hierarchical parallel systems modeled as HPPNs. It integrates key elements of HPPNs such as portal transitions, complex and compound tokens, and transition types specific to adaptive systems.

One of ScopeNet's strengths lies in embedding qualitative analysis within the modeling process, avoiding model transformation to other formalisms as LTL or CTL. This allows analysis of structure, behavior, and performance within the same framework. ScopeNet is used to define and simulate the HARMONY model, ensuring consistency between the formal specification and its operational behavior.

B. HPPN-BASED MODEL FOR HARMONY

The model reflects HARMONY's architectural hierarchy and encodes the adaptive decision-making logic that spans across local and global control levels. Through the HPPN structure, we model the autonomous behavior of each microservice as well as their coordinated scaling under a central application manager, capturing how quality objectives are monitored, analyzed, and reconciled.

The model is composed of three hierarchical levels:

- A top-level **AM** responsible for global adaptation and resource orchestration.
- A set of **MSMs** modeling local control per microservice.
- Nested **QAP** modules that handle quality objectives adaptation.

Each level implements a tailored MAPE-K loop, encoded through structured tokens, specialized transitions, and coordinated interaction via portal firing.

1) APPLICATION MANAGER

The AM is responsible for initiating and coordinating adaptations based on the overall system performance. It continuously monitors system-wide metrics and orchestrates corrective actions whenever service-level objectives are violated. As illustrated in Figure 3, the AM operates through four main phases:

- **Monitoring:** Global system metrics are collected and encapsulated as `metric` tokens. These tokens are injected into the model through the initial portal `Monitor`.
- **Analysis:** The `Analyzer` transition evaluates metric tokens against expected thresholds. If violations are detected, adaptation requests are formulated and forwarded to MSMs via the portal `Auto_scaling`.
- **Planning:** Input `MSM_plan` tokens from MSMs are received as a compound token, which is decomposed using `Fissioner` into single plans. They are then passed to the `Compromiser`, which computes a Pareto-optimal compromise plan to balance global objectives.
- **Execution:** Final adaptation decisions are committed via the `Executor`, which updates the `Deployment` place by adjusting the number of replicas for each microservice included in the computed plan.

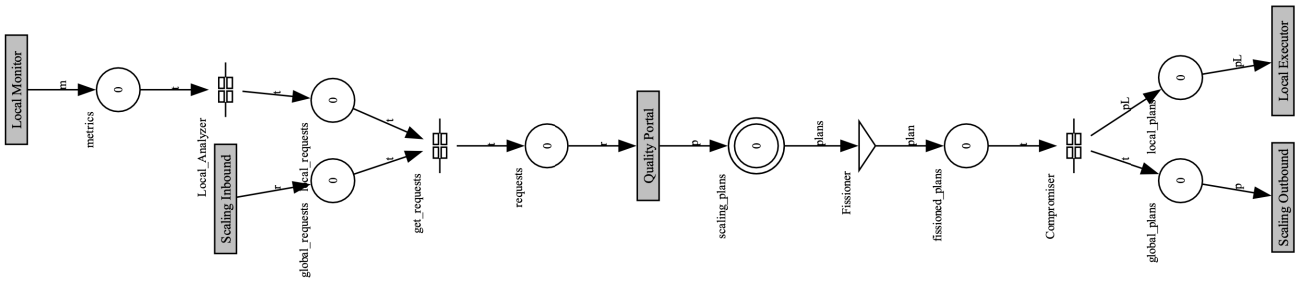


FIGURE 4. Formal HPPN model of a microservice manager.

We use *computing transitions* to implement both the `Analyzer` and the `Compromiser` components within the HPPN model. These transitions are responsible for performing global threshold evaluations and executing the global adaptation plan selection algorithms, including the subset filtering and Pareto-based compromise strategy described in Section IV-A2.

2) MICROSERVICE MANAGER

As illustrated in Figure 4, the MSM loop includes:

- **Monitoring:** The `Monitor` transition processes local metrics and shifts tokens to the evaluation phase. Meanwhile, the `Scaling Inbound` receives global adaptation requests.
- **Analysis:** The `Analyzer` evaluates both internal and external metrics of the system. Depending on the origin of the trigger (local or global), the MSM applies either local adaptation or delegates refinement to its internal QAPs through the `Quality Portal`.
- **Planning:** Once QAP plans are returned and extracted by the `Fissioner`, the `Compromiser` scores the results. If the adaptation was locally triggered, the result is immediately executed; otherwise, it is returned to the AM for coordination.
- **Execution:** The `Executor` applies scaling actions locally by adjusting its number of replicas in the `Deployment`, while the `Scaling Outbound` outputs a plan to be merged globally.

Within the MSMs, the `Analyzer` is realized through two computing transitions: `local_analyzer` and `get_request`. The `local_analyzer` performs evaluations against local thresholds, while the `get_request` transition generates request tokens and classifies them as local or global based on the triggering violation. The `Compromiser` executes the adaptation decision process, as detailed in Algorithm 3, implementing the *plan selection* phase of the MSM Adaptation Decision Process.

3) QUALITY ANALYZE-PLAN (QAP)

As shown in Figure 5, each QAP is composed of:

- **Analysis and Planning:** The QAP receives tokens from the MSM through the `Quality_In` portal, classifies them as either optimization requests or local violations, and initiates the corresponding scaling procedure. The `Scale_to_Satisfy` computing transition implements Algorithm 4 to rapidly satisfy SLA thresholds, whereas `Scale_to_Optimize` implements Algorithm 5 to further enhance performance when the microservice is stable but the overall system is not. The resulting adaptation decisions are returned via the `Quality_Out` portal.

4) SHARED KNOWLEDGE

In the HPPN model, shared knowledge is represented by a shared `Deployment` place, accessible to all hierarchical components, including the AM, MSMs, and QAPs. The place maintains compound tokens encoding the real-time deployment state of each microservice, encompassing the current number of replicas and per-replica resource requirements, such as CPU, memory, and input data rate.

Beyond runtime configuration, this centralized token repository also represents shared adaptation context, enabling communication of planned changes, quality thresholds, and system-wide constraints. By synchronizing this information across hierarchical levels, the model ensures coherent decision-making, alignment between local and global adaptation goals, and consistent execution of coordinated adaptations. The model is implemented using the ScopeNet framework.

a: MODELING ASSUMPTIONS

The HPPN-based model represents adaptation logic at the decision-making level. Communication between components is therefore abstracted as atomic synchronization at portal transitions, neglecting modeling transmission latency or concurrent message interleavings. Similarly, system load evolution is represented using monitored metric values rather than continuous or stochastic arrival processes. These abstractions follow standard practice in hierarchical formal modeling and allow the verification process to focus on the correctness and consistency of scaling decisions. They do not restrict applicability: timing delays, elastic response lag, and performance variability can be incorporated by enriching token data and guard predicates, without need for extending the underlying HPPN semantics. The evaluation presented

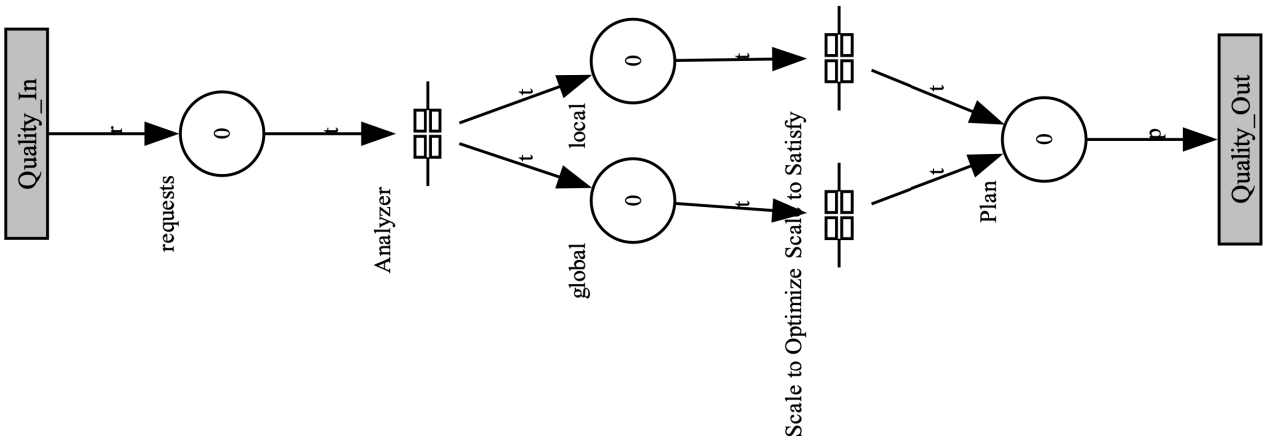


FIGURE 5. Formal HPPN model of a Quality Analyze-Plan (QAP) module.

in Section VI examines the behavior of the approach under realistic runtime conditions.

Practical Deployment Considerations. The abstraction choices adopted in the HPPN-based model focus on decision correctness and coordination rather than low-level execution timing. In practical deployments, such as Kubernetes or other cloud-native orchestration platforms, the proposed Application Manager, Microservice Managers, and Quality Analyze-Plan modules can be realized as controllers or operators interacting asynchronously through monitoring pipelines and control APIs. Runtime uncertainties such as metric noise, delayed observations, and asynchronous execution may affect adaptation responsiveness but do not compromise the correctness of the decision logic. These effects can be mitigated through common mechanisms including sliding-window aggregation, conservative threshold margins, and bounded adaptation frequencies. Importantly, such deployment-specific concerns can be incorporated by enriching token data and guard conditions without modifying the underlying hierarchical adaptation logic.

C. IMPLEMENTING HARMONY WITHIN THE ScopeNet FRAMEWORK

The auto-scaling architecture of the HARMONY model was implemented through three primary classes in ScopeNet: `AppManager`, `MsManager`, and `QaP`. These classes encode the structural design of the model, including the parent-child relationships, portals, and knowledge sharing mechanisms between the application manager and microservice managers.

- The `AppManager` class models the application-level manager, responsible for global metric monitoring, violation analysis, coordinated plan selection, and execution.
- The `MsManager` class represents each microservice manager, handling local metric analysis, local adaptation, and communication with its QAP components.
- The `QaP` class implements the quality-specific adaptation modules for each microservice, evaluating whether to apply *scale-to-satisfy* or *scale-to-optimize* strategies.

Adaptation strategies and decision-making processes are implemented as dedicated methods within ScopeNet's `ComputingTransition` class. Each computing transition defined in the HARMONY model is associated with its corresponding method upon instantiation, enabling the transition to execute a specific algorithm when triggered, including:

- Threshold verification for resource violations.
- Local compromise and multi-objective plan selection.
- Pareto-based optimization for global compromise decisions.

Combining the architectural classes with the algorithmic methods within computing transitions fully realizes the HARMONY auto-scaling model, enabling dynamic adaptation based on real-time system states.

D. SCALABILITY ANALYSIS OF THE HPPN-BASED MODEL

1) STATE-SPACE COMPLEXITY

Let R_{AM} denotes the reachability graph of the Application Manager subnet, R_{MSM_i} that of the i-th Microservice Manager, and $R_{QAP_{i,j}}$ that of the j-th QAP module associated with MSM_i.

A flat Petri net representation that encodes the entire system in a single net would induce a global state space of the form

$$R_{\text{flat}} \approx \prod_i R_{MSM_i} \cdot \prod_{i,j} R_{QAP_{i,j}},$$

which grows combinatorially as the number of services and quality modules increases.

In contrast, the HPPN representation preserves behavioral independence across hierarchical levels. Each subnet evolves

according to its own marking, and interaction occurs only through structured token exchange via portal transitions. The resulting reachability structure is therefore *hierarchical*:

$$R_{\text{hier}} \approx R_{AM} + \sum_i R_{MSM_i} + \sum_{i,j} R_{QAP_{i,j}}.$$

Thus, state-space growth is additive rather than multiplicative, enabling the model to avoid the state-space explosion typically associated with monolithic Petri net encodings of distributed adaptation logic.

2) COMPUTATIONAL COMPLEXITY

Let m denotes the number of microservices and n_i the number of feasible scaling configurations evaluated in the QAP of MSM_i. The analysis and planning procedure inside each QAP involves exploring a bounded configuration space, giving a per-module cost

$$T_{\text{QAP}_i} = O(n_i).$$

At the application level, coordination among MSMs is performed by extracting a Pareto frontier over the m candidate plans returned from the MSMs. The compromise selection is carried out using a dominance-based filtering strategy with computational cost

$$T_{\text{AM}} = O(m \log m).$$

The total adaptation complexity is therefore

$$T_{\text{adapt}} = \sum_{i=1}^{m} O(n_i) + O(m \log m).$$

If the size of each local configuration space is bounded by a constant $n_{\max}$, then

$$T_{\text{adapt}} = O(mn_{\max} + m \log m),$$

yielding adaptation overhead that scales approximately linearly with the number of microservices.

3) COMPOSITIONAL SCALABILITY

Scalability of the model arises from its compositional structure. Each MSM and its associated QAP modules maintain localized control and operate over restricted state representations. The AM coordinates adaptation by exchanging compact summaries of local decisions in the form of complex or compound tokens, rather than full local state.

The decomposition ensures:

- reasoning and decision-making remain local to each subnet,
- hierarchical coordination does not require global state enumeration, and
- increases in the number of services result in proportional, rather than combinatorial, growth of the overall model.

Thus, scalability is achieved structurally through hierarchical separation of concerns, and the model remains tractable as the number of microservices and quality-control modules increase.

Scalability Interpretation. The above analysis demonstrates that HARMONY achieves scalability by construction. Unlike flat or centralized auto-scaling models, whose state space and decision complexity grow combinatorially with the number of services, the hierarchical HPPN structure preserves behavioral independence across layers. Local reasoning remains confined to Microservice Managers and their Quality Analyze-Plan modules, while global coordination is performed through compact plan summaries exchanged with the Application Manager. As a result, both the state-space size and the adaptation overhead grow approximately linearly with the number of microservices under bounded local configuration spaces, ensuring practical feasibility for large-scale and complex service graphs.

VI. CASE STUDY: AUTO-SCALING IN AN E-BUSINESS PLATFORM

To evaluate the effectiveness of HARMONY, we apply it to a representative e-business platform composed of multiple interacting microservices. E-business platforms are particularly challenging environments for auto-scaling due to their highly dynamic workloads, fluctuating traffic patterns, and the need to satisfy diverse quality requirements simultaneously, including performance, cost efficiency, reliability, and security.

The e-business case study offers a realistic and complex system to illustrate how HARMONY's hierarchical, multi-objective, and formally verifiable framework effectively coordinates local and global scaling decisions while balancing trade-offs among competing system quality attributes.

While the presented case study focuses on a platform composed of six microservices for clarity and traceability, the proposed approach is not limited to this scale. The hierarchical architecture of HARMONY and the compositional scalability properties established in Section V-D ensure that the same adaptation logic applies to larger microservice ecosystems without introducing exponential growth in coordination or decision-making complexity.

A. E-BUSINESS PLATFORM ARCHITECTURE

The case study focuses on an e-business platform composed of six core microservices, each one with distinct operational roles, resource profiles, and quality priorities. The platform is structured following HARMONY's hierarchical auto-scaling architecture, which integrates local microservice managers with centralized coordination and shared knowledge.

Microservices composing the platform are:

- **Product Catalog**: Provides product listing, category browsing, and search functionalities.
- **Shopping Cart**: Manages user selections, shopping sessions, and cart updates.
- **Payment Service**: Handles secure payment transactions and transaction processing.

- **User Management**: Manages authentication, user profiles, and security policies.
- **Inventory Service**: Synchronizes stock levels and ensures real-time inventory consistency.
- **Order Management**: Processes order creation, validation, and tracking.

Each microservice is managed by a dedicated **MSM** that monitors local resource utilization (CPU and memory) and oversees quality-specific objectives through dedicated **QAP** modules. The QAPs are responsible for evaluating local quality violations and proposing adaptation plans to the MSM.

The platform's **AM** supervises system-wide metrics and consolidates adaptation plans from all MSMs to maintain global system qualities.

The **Shared Knowledge** component, implemented as a centralized `Deployment` repository, is continuously updated by all MSMs and the AM to maintain a consistent view of the deployment state, resource allocations, and adaptation plans across the system layers. The shared knowledge ensures that local and global decisions remain synchronized and that cross-service adaptations do not produce conflicting states.

B. QUALITY REQUIREMENTS AND SCALING METRICS

The global system qualities include **Performance** and **Cost**, and the metric thresholds are set to 25%–85% for CPU and 30%–80% for memory.

Each microservice monitors local CPU and memory utilization based on defined thresholds and according to specific local quality requirements that guide its scaling behavior:

- *Product Catalog:* Prioritizes Scalability, Availability, and Cost.
- *Shopping Cart:* Focuses on Performance, Latency, and Cost.
- *Payment Service:* Emphasizes Security and Latency.
- *User Management:* Ensures Security and Availability.
- *Inventory Service:* Balances Consistency and Performance.
- *Order Management:* Targets Responsiveness, Scalability, and Accuracy.

Table 3 summarizes the quality requirements and scaling thresholds for each microservice.

1) IMPLEMENTING THE E-BUSINESS PLATFORM IN ScopeNet

For the e-business case study, each MSM was instantiated as a child net of the AM using ScopeNet's `MsManager` class (see Figure 4), with its quality requirements and resource thresholds directly encoded in the model. The AM was instantiated using the `AppManager` class (see Figure 3), managing the global monitoring, analysis, plan selection, and execution portals.

QAP components are instantiated as individual HPPN child nets within each microservice (see Figure 5), capturing the specific quality requirements of each service. Each QAP is connected to its corresponding microservice via portals, according to the hierarchical structure defined in the architectural model.

Resource violations are injected into the system using tokens directly placed in the monitoring components of the AM and MSMs. The system autonomously propagated these violations throughout the hierarchical structure, triggering both local and global adaptations in line with the defined thresholds and quality weights.

C. MODEL EXECUTION AND RESULTS

To evaluate the HARMONY framework, we performed a series of controlled model executions on the instantiated e-business platform. The objective is to validate the system's capacity to autonomously detect resource violations, coordinate local and global scaling actions, and achieve balanced multi-objective adaptations under diverse scenarios.

1) SCENARIO 01: GLOBAL CPU OVERLOAD

We simulate a global CPU overload by injecting a violation token with a CPU value of 92%, exceeding system threshold of 85%. The adaptation objective is to reduce CPU usage by at least 7%, bringing it below the acceptable limit. Through this scenario, we aim to evaluate whether the HARMONY model can (1) bring the system metric back within threshold bounds and (2) select the most effective adaptation plans in terms of performance and cost efficiency through Pareto optimization.

Upon detecting a violation, the AM triggers a coordinated adaptation cycle. Each MSM receives the adaptation request and responds by generating a local plan that recommends an increase in replicas, accompanied by its estimated impact on CPU consumption reduction, performance enhancement, and cost overhead. Analysis of the execution trace confirms that each QAP employs the `scale_to_optimize` transition to generate its adaptation plan:

```
Trace d'execution: ['Analyzer', 'Scale to
    Optimize', 'Quality Out']
```

Table 4 shows that each QAP complies with the replica constraints of its microservice. For example, the Product Catalog, initially deployed with 3 replicas, proposes adding 2 replicas, remaining within its maximum of 5. All services demonstrate similar adherence to their respective bounds.

The AM aggregates all candidate plans and executes the Pareto optimization procedure, which filters valid plan combinations—those achieving the target 7% CPU reduction—and selects a non-dominated subset that optimally balances performance and cost. Figure 6 depicts all valid combinations and highlights the selected plan set. The chosen subset includes plans from the Product Catalog

TABLE 3. Quality requirements and resource thresholds of e-business microservices.

Microservice	CPU Threshold (%)	Memory Threshold (%)	Quality Requirements (Weights)
Product Catalog	20 – 70	30 – 80	Scalability (0.5), Availability (0.3), Cost (0.2)
Shopping Cart	25 – 75	35 – 85	Performance (0.4), Latency (0.5), Cost (0.1)
Payment Service	30 – 80	40 – 90	Security (0.7), Latency (0.3)
User Management	15 – 60	25 – 75	Security (0.6), Availability (0.4)
Inventory Service	20 – 65	30 – 80	Consistency (0.5), Performance (0.5)
Order Management	10 – 50	20 – 60	Responsiveness (0.4), Accuracy (0.3), Scalability (0.3)

TABLE 4. Suggested adaptation plans from the microservices.

Microservice	Replicas	Metric Impact	Performance	Cost
Inventory Service	2	5.0	8.0	4.0
Order Management	1	2.5	4.0	2.0
Product Catalog	2	5.0	8.0	4.0
Shopping Cart	2	5.0	8.0	4.0
Payment Service	1	2.5	4.0	2.0
User Management	1	2.5	4.0	2.0

and Order Management services, achieving the adaptation objective while providing a balanced trade-off between cost and performance:

```
Selected Plans: {'id': 'Product Catalog', '
    plan': 2, 'metric_impact': 5.0, '
    performance': 8.0, 'cost': 4.0}
{'id': 'Order Management', 'plan': 1, '
    metric_impact': 2.5, 'performance': 4.0,
    'cost': 2.0}
```

Finally, the shared knowledge state confirms the updated replica counts for Product Catalog and Order Management:

```
{
  'ms_knowledge': [
    {'ms_id': 'Product Catalog',
     'current_replicas':5,
     'CPU': 3.13,
     'Memory': 0.78,
     'performance': 4.0,
     'cost': 2.0},

    {'ms_id': 'Shopping Cart',
     'current_replicas': 4,
     'CPU': 3.13,
     'Memory': 0.94,
     'performance': 5.0,
     'cost': 2.5},

    {'ms_id': 'Payment Service',
     'current_replicas': 2,
     'CPU': 3.75,
     'Memory': 0.94,
     'performance': 6.0,
     'cost': 3.0},

    {'ms_id': 'User Management',
     'current_replicas': 2,
     'CPU': 2.5,
     'Memory': 0.63,
     'performance': 3.5,
     'cost': 1.8},

    {'ms_id': 'Inventory Service',
     'current_replicas': 2,
     'CPU': 3.13,
     'Memory': 0.94,
     'performance': 4.5,
     'cost': 2.2},

    {'ms_id': 'Order Management',
     'current_replicas': 3,
     'CPU': 2.5,
     'Memory': 0.78,
     'performance': 4.0,
     'cost': 2.0}
  ]
}
```

We notice that the adaptation not only resolved the violation but also achieved measurable performance gains with only a minimal increase in cost, thereby validating the effectiveness of the HARMONY strategy.

2) SCENARIO 02: LOCAL MEMORY UNDERLOAD IN SHOPPING CART

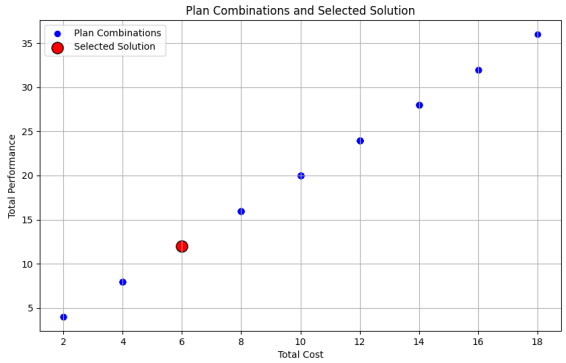
In this scenario, we simulate a local memory underload in the Shopping Cart microservice, with utilization dropping to 20%, significantly below the 35% threshold. Upon detection, the Shopping Cart MSM triggers its local adaptation loop, activating the Cost, Latency, and Performance QAPs to evaluate resource reduction while maintaining service quality.

The execution trace indicates that the QAPs employ the `scale_to_satisfy` algorithm to determine the maximum number of replicas that can be safely removed without breaching quality constraints. Starting with 5 replicas, the function iteratively decreases the replicas, estimating the impact on the corresponding quality attributes and memory utilization at each step:

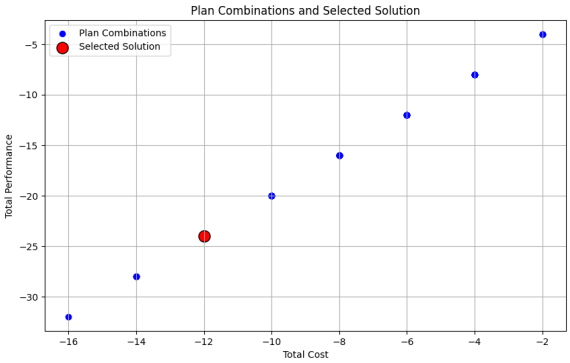
```
Trace d'execution: ['Analyzer', 'Scale to
    Satisfy', 'Quality_Out']
```

After firing, the QAPs responded with the following tokens indicating the proposed adaptation actions:

```
Returned Token:  Quality Portal
    CompoundToken({{'metric': 'Memory', '
    value': 35.0, 'label': 'local', '
    current_replicas': 4, 'mode': -1, '
    violation': 15, 'plan':
    -3, 'quality': 2.0, 'qap': 'Cost'},
{'metric': 'Memory', 'value': 25.0, 'label':
     'local', 'current_replicas': 4, 'mode':
```



(a) Pareto front for Scenario 01 (Scale-Up).



(b) Pareto front for Scenario 03 (Scale-Down).

FIGURE 6. **Pareto front visualization.**

```
    -1, 'violation': 15,  'plan':
    -1, 'quality': 0.76, 'qap': 'Latency'},
{'metric': 'Memory', 'value': 35.0, 'label':
     'local', 'current_replicas': 4, 'mode':
    -1, 'violation': 15,  'plan':
    -3, 'quality': 6.53, 'qap': 'Performance
    '}})
```

Both the Cost and Performance QAPs proposed scaling down by 3 replicas, while the Latency QAP recommended reducing only one replica, as more scaling would provoke violation of the quality value threshold.

After aggregating these plans, the local compromiser applied the scoring function to compute the optimal adaptation plan. The final plan suggests reducing replicas from 4 to 2, reflecting a trade-off between memory adaptation and latency preservation:

```
FINAL TOKEN {'id': 'Shopping Cart', 'metric
    ': 'Memory', 'plan': -2, 'label': 'local
    '}
```

This scenario demonstrates HARMONY's ability to scale down microservices during resource underutilization. By reducing replicas while preserving critical quality attributes, HARMONY ensures cost efficiency without compromising service reliability. The execution trace and shared knowledge confirm that the plan is generated and executed locally and autonomously, underscoring the architecture's decentralized responsiveness.

```
Trace d'execution: ['Quality Portal', '
    Fissioner', 'Compromiser', 'Local
    Executor']


[{'ms_id': 'Shopping Cart', '
    current_replicas':2, 'replicas_limit': 6,
     'metric_per_replicas': {'CPU': 0.5, '
    Memory': 0.6}, 'thresholds': {'CPU': (25,
     75), 'Memory': (35, 85)}, '
    quality_weights': {'Performance': 0.4, '
    Latency': 0.5, 'Cost': 0.1}, '
    current_quality': {'Performance': 7.68, '
    Latency': 0.43, 'Cost': 4}, '
    quality_limits': {'Performance': 0.6, '
    Latency': 0.7, 'Cost': 12}}]
```

3) SCENARIO 03: GLOBAL CPU UNDERLOAD WITH LOCAL CONFLICT

In this scenario, we inject a global CPU underload token with a value of 10%, triggering a scale-down objective at the system level. Simultaneously, the User Management microservice detects a local CPU overload at 90%, prompting it to increase its replicas to restore local performance. This setup evaluates HARMONY's ability to (1) coordinate conflicting adaptation needs across layers and (2) resolve global adaptation through coordinated down-scaling while preserving local service adaptation constraints.

Upon detection of the violation, the AM launches a system-wide adaptation cycle. At the same time, User Management independently executes its local pipeline, firing the full Quality Analysis and Planning sequence, which includes both local execution and global contribution:

```
Trace d'execution: ['Quality Portal', '
    Quality Portal',
'Fissioner', 'Fissioner',
'Compromiser', 'Compromiser',
'Local Executor', 'Scaling Outbound']
```

The local plan suggests adding 2 replicas to resolve the local violation:

```
FINAL TOKEN {'id': 'User Management', '
    metric': 'CPU', 'plan': 2, 'label': '
    local'}
```

Simultaneously, regarding the AM's adaptation objective, the User Management service proposes removing a unique replica, maintaining its required minimum to ensure continued compliance. The complete set of plans submitted to the AM is as follows:

```
{'id': 'Order Management', 'plan': -1, '
    metric': 'CPU', 'label': 'global'}
```

```
{'id': 'Product Catalog', 'plan': -1, '
    metric': 'CPU', 'label': 'global'}
{'id': 'Inventory Service', 'plan': -2, '
    metric': 'CPU', 'label': 'global'}
{'id': 'User Management', 'plan': -1, '
    metric': 'CPU', 'label': 'global'}
{'id': 'Payment Service', 'plan': -1, '
    metric': 'CPU', 'label': 'global'}
{'id': 'Shopping Cart', 'plan': -2, 'metric
    ': 'CPU', 'label': 'global'}
```


After collecting these plans, the AM executes Pareto optimization in scale-down mode, selecting combinations that achieve sufficient CPU reduction while balancing trade-offs between performance and cost. Figure 6b visualizes all valid combinations, highlighting the selected subset:


```
{'id': 'Order Management', 'plan': -1, '
    metric_impact': -2.5,
'performance': -4.0, 'cost': -2.0}
{'id': 'Product Catalog', 'plan': -1, '
    metric_impact': -2.5,
'performance': -4.0, 'cost': -2.0}
{'id': 'Inventory Service', 'plan': -2, '
    metric_impact': -5.0,
'performance': -8.0, 'cost': -4.0}
{'id': 'Shopping Cart', 'plan': -2, '
    metric_impact': -5.0,
'performance': -8.0, 'cost': -4.0}
```


This case confirms that HARMONY can simultaneously scale up and down across layers, enabling microservices to safeguard their local qualities while contributing to system-wide goals through globally coordinated decisions.

Figures 6a and 6b illustrate the Pareto fronts observed in the scale-up and scale-down scenarios, respectively. In Scenario 01, trade-offs occur in the positive domain, with performance gains achieved at the expense of increased resource usage. In Scenario 03, all values are negative, as the system reduces replicas to cut costs, which also slightly degrades performance. In both cases, HARMONY identifies plans that balance competing objectives, minimizing cost during scale-up and preserving performance during scale-down, while consistently achieving the adaptation goal. These results demonstrate the model's context-sensitive decision-making and its ability to effectively navigate multi-objective trade-offs.

4) SCENARIO 04

This scenario simulates a severe global CPU overload, requiring maximum adaptation from all microservices. The adaptation goal exceeds the combined capacity of standard plans, forcing each service to leverage its full scaling capability. Each microservice generates a local plan proposing the maximum feasible number of additional replicas within its allowed limits. The proposed plans are as follows:


```
{'id': 'Inventory Service', 'plan': 3, '
    metric_impact': 7.5, 'performance': 12.0,
     'cost': 6.0}
```

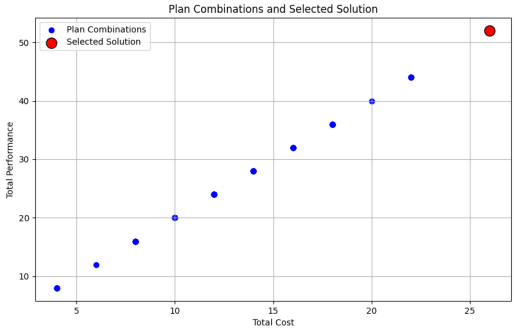



FIGURE 7. Pareto front visualization for Scenario 04.


```
{'id': 'Payment Service', 'plan': 2, '
    metric_impact': 5.0, 'performance': 8.0,
    'cost': 4.0}
{'id': 'Order Management', 'plan': 2, '
    metric_impact': 5.0, 'performance': 8.0,
    'cost': 4.0}
{'id': 'Shopping Cart', 'plan': 2, '
    metric_impact': 5.0, 'performance': 8.0,
    'cost': 4.0}
{'id': 'Product Catalog', 'plan': 2, '
    metric_impact': 5.0, 'performance': 8.0,
    'cost': 4.0}
{'id': 'User Management', 'plan': 2, '
    metric_impact': 5.0, 'performance': 8.0,
    'cost': 4.0}
```


The AM collects all proposed plans and triggers Pareto optimization in scale-up mode, aiming to balance performance gains and cost increase. Figure 7 shows all candidate combinations and highlights the selected solution, which lies on the extreme corner of the Pareto front, confirming that the system opted for maximal adaptation due to high CPU pressure.

Upon execution, the shared knowledge confirms that each microservice has reached its replica limit, ensuring full utilization of available resources to resolve the global violation:


```
'ms_knowledge': [
{'ms_id': 'Product Catalog', '
    current_replicas': 5 \ldots},
{'ms_id': 'Shopping Cart', 'current_replicas
    ': 6 \ldots},
{'ms_id': 'Payment Service', '
    current_replicas': 4 \ldots},
{'ms_id': 'User Management', '
    current_replicas': 4 \ldots},
{'ms_id': 'Inventory Service', '
    current_replicas': 5 \ldots},
{'ms_id': 'Order Management', '
    current_replicas': 4 \ldots}
]
```


This scenario validates HARMONY's ability to maximize adaptation within system constraints and confirms that the model respects replica limits while resolving high-severity violations.

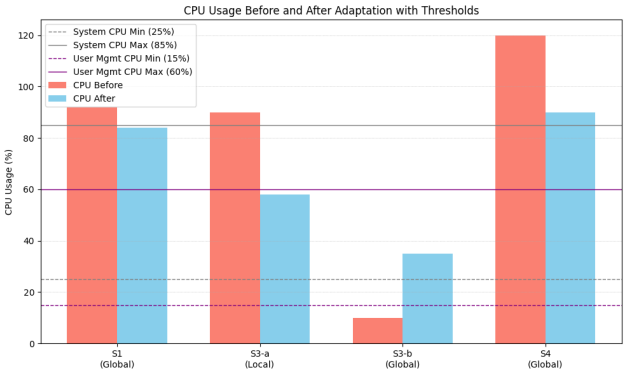


FIGURE 8. CPU usage before and after adaptation across scenarios.

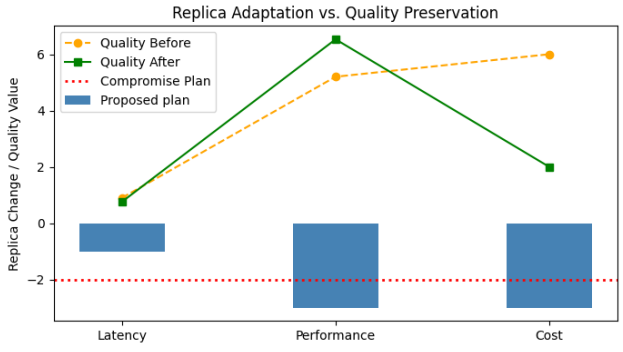


FIGURE 9. Replica adaptation vs. quality preservation. The graph shows the individual quality responses and the compromise plan derived by the local QAPs.

D. DISCUSSION

The experimental scenarios demonstrate the ability of HARMONY to perform adaptive and coordinated scaling actions in response to diverse system conditions. Across all use cases, the model successfully restored metrics within defined thresholds while balancing competing quality requirements.

Through hierarchical decision making and modular QAP components, HARMONY ensures that both global and local adaptation requests are addressed without compromising the overall system stability. In Scenario 01, the system scaled up selectively, prioritizing the microservices that contribute most, according to Pareto optimization. In Scenario 02, a local under-utilization was resolved autonomously, showing that microservices can scale down without centralized intervention. Scenario 03 validated the model's coordination logic, allowing a microservice to scale up locally while still contributing to a global scale-down objective. Scenario 04 further confirmed that when nearing capacity limits, HARMONY is capable of generating effective full-scale adaptations.

One key strength of HARMONY lies in its integrated support for multi-objective trade-offs. Rather than relying on a single optimization goal, the model adjusts the plan selection logic based on the adaptation context, balancing cost efficiency during scale-up with quality preservation during scale-down. At the system level, this behavior is reflected in the Pareto fronts generated in different scenarios (see Figures 6a and 6b), where the selected plan sets consistently offer the most balanced cost–performance trade-offs.

From a local viewpoint, each microservice's QAP tailors its adaptation plans based on service-specific quality limits. As illustrated in Figure 9, the compromise plan integrates proposals from multiple QAPs, each with distinct quality sensitivities, to determine an adaptation that preserves local quality while optimizing resource usage. This confirms that HARMONY's hierarchical structure enables multi-layered optimization, where system-wide goals are met without overriding critical local constraints.

Moreover, Figure 8 illustrates HARMONY's precision in regulating resource usage across scenarios. The system not only restores global metrics within defined thresholds but also adheres to local constraints for each microservice, demonstrating its ability to balance metric enforcement with policy compliance across hierarchical adaptation layers. An additional strength of HARMONY is the use of shared knowledge as a coordination mechanism. By maintaining a centralized deployment state, the framework ensures that all decisions are based on the system current state, preventing conflicts and facilitating informed trade-offs.

Although the model demonstrates robust adaptability and responsiveness, its performance relies on accurate prediction functions and well-defined quality thresholds. Misconfigured or rigid parameters may lead to over- or under-scaling. Future work could integrate runtime learning to dynamically refine these parameters. Moreover, while deployment-imposed scaling limits were respected in the case study, they may be bottlenecks in real-world environments if not continuously monitored and adjusted.

VII. CONCLUSION

In this paper, we presented HARMONY, a hierarchical and formally verifiable framework for auto-scaling in microservices-based cloud environments. Unlike traditional approaches that emphasize either local autonomy or centralized control, HARMONY integrates both through a layered MAPE-K architecture supported by a shared knowledge model, enabling coherent and scalable adaptation decisions.

At the strategy level, HARMONY employs Pareto-based optimization to reconcile multiple and often conflicting system qualities, including performance, cost, and reliability. This compromise-driven approach ensures that no quality dimension is disproportionately sacrificed, even under resource constraints or competing adaptation triggers. The framework is fully formalized using Hierarchical Parallel Petri Nets (HPPNs), providing simulation and verification capabilities that guarantee correctness, boundedness, and coordination soundness.

A realistic case study on an e-business platform demonstrated HARMONY's effectiveness across diverse adaptation scenarios, ranging from local under-utilization to global overload, while preserving both system-wide objectives and local quality constraints. Results confirmed its ability to coordinate scalable adaptations, select non-dominated plan sets, and maintain consistency between local and global behaviors.

By combining formal modeling, multi-objective adaptation, and hierarchical coordination, HARMONY establishes a robust foundation for building resilient, scalable, and self-adaptive cloud systems. Future work will extend the framework with learning-based quality predictors, explore runtime adaptation refinement, and apply HARMONY to emerging domains such as edge computing and federated microservice environments.

REFERENCES

[1] V. L. Nogueira, F. S. Felizardo, A. M. M. M. Amaral, W. K. G. Assunção, and T. E. Colanzi, "Insights on microservice architecture through the eyes of industry practitioners," in *Proc. IEEE Int. Conf. Softw. Maintenance Evol. (ICSME)*, Oct. 2024, pp. 765–777.

[2] J. P. K. S. Nunes, S. Nejati, M. Sabetzadeh, and E. Y. Nakagawa, "Self-adaptive, requirements-driven autoscaling of microservices," in *Proc. 19th Int. Symp. Softw. Eng. Adapt. Self-Manag. Syst. (SEAMS)*, Lisbon, Portugal. New York, NY, USA: Association for Computing Machinery, 2024, pp. 168–174, doi: 10.1145/3643915.3644094.

[3] H. Ahmad, C. Treude, M. Wagner, and C. Szabo, "Smart HPA: A resource-efficient horizontal pod auto-scaler for microservice architectures," in *Proc. IEEE 21st Int. Conf. Softw. Archit. (ICSA)*, Jun. 2024, pp. 46–57.

[4] S. Xie, J. Wang, B. Li, Z. Zhang, D. Li, and P. C. K. Hung, "PBScaler: A bottleneck-aware autoscaling framework for microservice-based applications," *IEEE Trans. Services Comput.*, vol. 17, no. 2, pp. 604–616, Mar. 2024, doi: 10.1109/TSC.2024.3376202.

[5] C. Meng, S. Song, H. Tong, M. Pan, and Y. Yu, "DeepScaler: Holistic autoscaling for microservices based on spatiotemporal GNN with adaptive graph learning," in *Proc. 38th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Sep. 2023, pp. 53–65.

[6] A. Horn, H. M. Fard, and F. Wolf, "Multi-objective hybrid autoscaling of microservices in kubernetes clusters," in *Proc. 28th Eur. Conf. Parallel Process.* Cham, Switzerland: Springer, 2022, pp. 233–250.

[7] S. N. A. Jawaddi, M. H. Johari, and A. Ismail, "A review of microservices autoscaling with formal verification perspective," *Softw., Pract. Exper.*, vol. 52, no. 11, pp. 2476–2495, Nov. 2022.

[8] D. R. Augustyn, Ł. Wyciślik, and M. Sojka, "Tuning a kubernetes horizontal pod autoscaler for meeting performance and load demands in cloud deployments," *Appl. Sci.*, vol. 14, no. 2, p. 646, Jan. 2024.

[9] V. M. Sanjay, S. Kyalkond, and A. Indra Kumar, "Analysis of aws auto scaling strategy in cloud computing," *Int. J. Innov. Res. Sci. Eng. Technol.*, vol. 10, p. 8, Dec. 2021.

[10] A. Mishra, "Borg autoscaling at scale: Lessons from managing millions of jobs," in *Proc. 13th ACM Symp. Cloud Comput. (SoCC)*, 2022, pp. 432–447.

[11] C. Curino, S. Krishnan, K. Karanasos, S. Rao, G. M. Fumarola, B. Huang, K. Chaliparambil, A. Suresh, Y. Chen, S. Heddaya, R. Burd, S. Sakalanaga, C. Douglas, B. Ramsey, and R. Ramakrishnan, "Hydra: A federated resource manager for data-center scale analytics," in *Proc. NSDI*, 2019, pp. 177–191.

[12] A. Ernstsson, "A pattern for decentralized autoscaling," *ACM Trans. Auto. Adapt. Syst.*, vol. 17, no. 2, pp. 1–25, 2022.

[13] W. Chen, "Decentralized autoscaling in edge computing: A SLA violation analysis," *IEEE Internet Things J.*, vol. 10, no. 4, pp. 3215–3230, Jan. 2023.

[14] Q. Zhang, "FIRM: Adaptive resource management for cloud microservices," in *Proc. IEEE 42nd Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Sep. 2022, pp. 1–11.

[15] H. Lee, "Hierarchical control plane for large-scale kubernetes clusters," in *Proc. ACM Symp. Cloud Comput. (SoCC)*, 2023, pp. 312–326.

[16] F. Rossi, V. Cardellini, and F. L. Presti,, "Hierarchical auto-scaling in kubernetes," in *Proc. IEEE Int. Conf. Autonomic Comput. Self-Organizing Syst. (ACSOS)*, Sep. 2020, pp. 28–37, doi: 10.1109/ACSOS49614.2020.00023.

[17] K. Alexander, M. Hanif, C. Lee, E. Kim, and S. Helal, "Cost-aware orchestration of applications over heterogeneous clouds," *PLoS ONE*, vol. 15, no. 2, Feb. 2020, Art. no. e0228086.

[18] Z. Xiao and S. Hu, "DScaler: A horizontal autoscaler of microservice based on deep reinforcement learning," in *Proc. 23rd Asia–Pacific Netw. Oper. Manage. Symp. (APNOMS)*, Sep. 2022, pp. 1–6.

[19] H. Yuan and S. Liao, "A time series-based approach to elastic kubernetes scaling," *Electronics*, vol. 13, no. 2, p. 285, Jan. 2024.

[20] M. Dashtbani and L. Tahvildari, "STaleX: A spatiotemporal-aware adaptive auto-scaling framework for microservices," 2025, *arXiv:2501.18734*.

[21] S. Naik and M. D'Souza, "Detection of faults in microservices using Petri nets," in *Proc. 16th Innov. Softw. Eng. Conf.*, NY, NY, USA, Feb. 2023, pp. 1–5.

[22] G. Giuliari, M. Wyss, M. Legner, and A. Perrig, *GMA: A Pareto Optimal Distributed Resource-Allocation Algorithm*. Cham, Switzerland: Springer, 2021, pp. 243–261.



SOUHEIR MERKOUCHE was born in Constantine, Algeria, in 1994. She received the master's degree in computer science from the University of Constantine 2-Abdelhamid Mehri, Constantine, in 2020, specializing in formal modeling of self-adaptive systems, where she is currently pursuing the Ph.D. degree in computer science.

She is a member of the LIRE Laboratory (Laboratoire d'Informatique et Réseaux) and has authored several papers presented at international conferences, covering topics, such as cloud computing, adaptive systems, and formal verification. Her thesis focuses on formal modeling and auto-scaling strategies for microservices-based architectures. She has contributed to collaborative research efforts on dynamic and distributed system management. She continues to explore rigorous modeling approaches for autonomous systems in cloud environments. Her current research interests include hierarchical control, multi-objective optimization, and the use of Petri nets for the formal specification of adaptive behaviors.



CHAFIA BOUANAKA received the Ph.D. degree in computer science from Mentouri University, Constantine, in 2010, and the Habilitation á Diriger des Recherches (H.D.R.) degree, in 2018.

She conducts her research at the LIRE Laboratory, where she is a member of the Software Engineering and Distributed Systems Team. She is currently a Full Professor with the University of Constantine 2-Abdelhamid Mehri, Algeria. Her research focuses on the formal specification and analysis of complex systems, with a particular interest in self-adaptive systems based on high-level Petri nets. More recently, she has been exploring the integration of machine learning techniques with formal methods to improve decision-making processes and maintain system quality under uncertainty.

Dr. Bouanaka has served as the Chair of the International Conference on Advanced Aspects of Software Engineering, since 2018. She is also an Active Reviewer for several peer-reviewed journals, including *Journal of King Saud University-Computer and Information Sciences* (Elsevier) and *Information and Software Technology* (Elsevier).



ELHADJ BENKHELIFA (Senior Member, IEEE) is currently a Full Professor of computer science and the Director of the Smart Systems, AI and Cybersecurity Research Centre, University of Staffordshire, U.K. He has published close to 200 articles in international conferences, journals, and chaired 10st of prominent international conferences. His research interests include biologically inspired algorithms, digital transformation, microservices, cloud/edge/mobile computing, AI, cybersecurity and resilience, software engineering, and data governance.

He is an expert member of several U.K. and European commission groups and committees. He is a member of the Alan Turing Institute's Knowledge Graph Working Group and the Director of the Knowledge Graph Alliance. He is the Ex-Chair and the Education Activities for IEEE U.K. and Ireland Section.



CHIRINE GHEDIRA GUEGAN received the Habilitation degree in computer science from University Claude Bernard Lyon 1.

She is currently a Full Professor of computer science with the iaelyon School of Management, Jean Moulin Lyon 3 University, and the Deputy Head of the Service Oriented Computing (SOC) Research Team, Lyon Research Center for Images and Intelligent Information Systems Laboratory associated with the French National Center for Scientific Research. She focused her work on the large-scale integration of heterogeneous data sources in different settings with attention to the servitization of sources, data privacy, and security. Her recent didactic and research activities cover the digital strategy, smart services for emergent disease detection, and integration of structured and unstructured resources in the health decision process. She has published more than 100 research articles in internationally recognized journals and congresses and has been the principal investigator of a number of collaborative projects.